Identifying Bottlenecks

find what breaks first, before production does

Watch the 4-min video

In one line: the bottleneck is the resource that limits throughput for the whole system; find it by writing peak load and capacity next to every component on the critical path, and the one with the highest load ÷ capacity ratio is it.

1 A system is only as fast as its narrowest point

Your design handles 100,000 requests per second on paper. Then launch day comes and everything slows to a crawl. Not because every part is too small, but because *one* part is.

A **bottleneck** is the resource that limits throughput for the whole system. A request passes through a load balancer, app servers, a cache and a database. If the database can take 5,000 writes per second, the whole write path tops out at 5,000, no matter how many app servers you add. **Scaling anything else just moves the queue.**

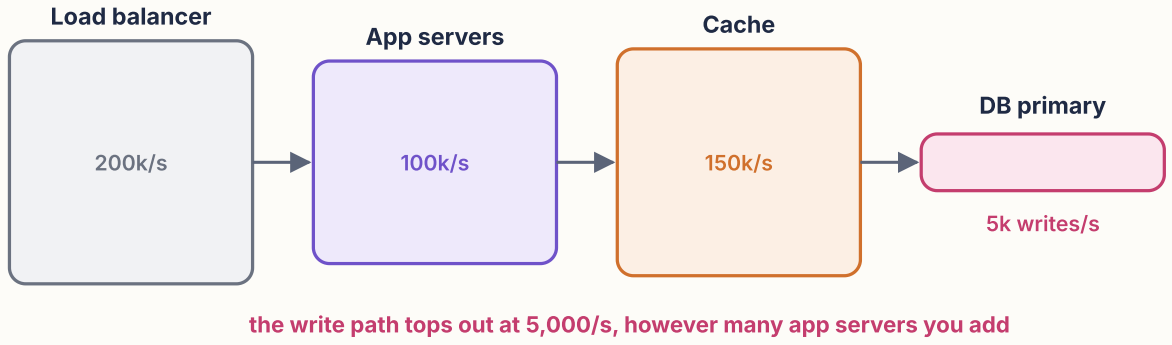


Figure 1. Throughput of a path is the minimum capacity along it. Here the single database primary is the narrow pipe.

2 Spotting one in a running system

Look for **saturation**: CPU pinned near 100%, disks maxed out, connection pools exhausted, or a queue whose depth keeps growing. Latency climbs sharply as a resource nears saturation, because requests start waiting in line. The resource that saturates first is the one to fix.

The **USE method** (from Brendan Gregg) makes this systematic: for every resource, check **U**tilization, **S**aturation and **E**rrors.

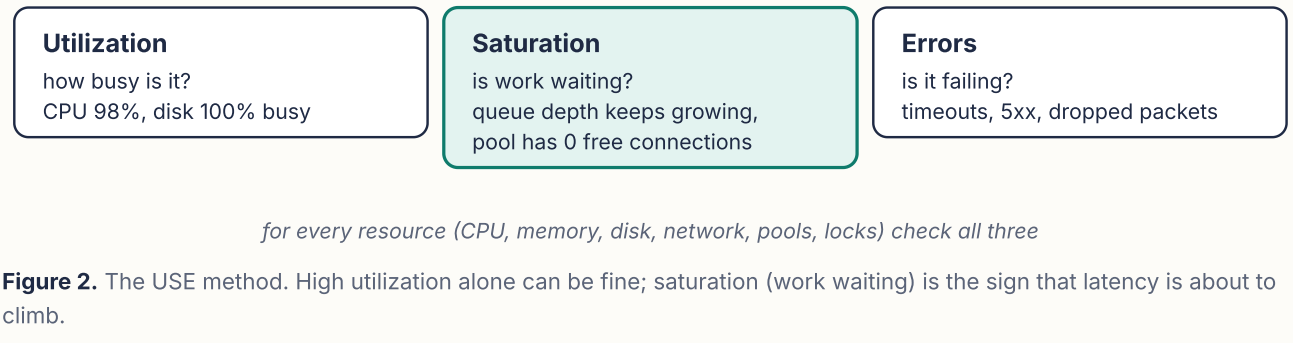


Figure 2. The USE method. High utilization alone can be fine; saturation (work waiting) is the sign that latency is about to climb.

3 The usual suspects in a design

Suspect	Why it limits	Typical fix
Database writes	A single primary takes every write.	Batch, buffer, shard
Hot partition	One key, like a celebrity account, gets far more traffic than the rest.	Split or salt the key, cache it
Network and egress	A link or NIC fills up, often with large payloads.	CDN, compression, smaller responses
Single consumers, global locks	One worker drains a queue, or one lock serializes everything.	Partition the work, remove the lock

4 Finding it in an interview: load ÷ capacity

Use your estimates. Walk the **critical path** of each major request. Next to every component, write the **peak load** it receives and the **capacity** it has. Divide one by the other. **The component with the highest ratio is your bottleneck.**

Remember that load concentrates: reads multiply through fan-out (one feed request may read 50 items), and peaks are several times the average.

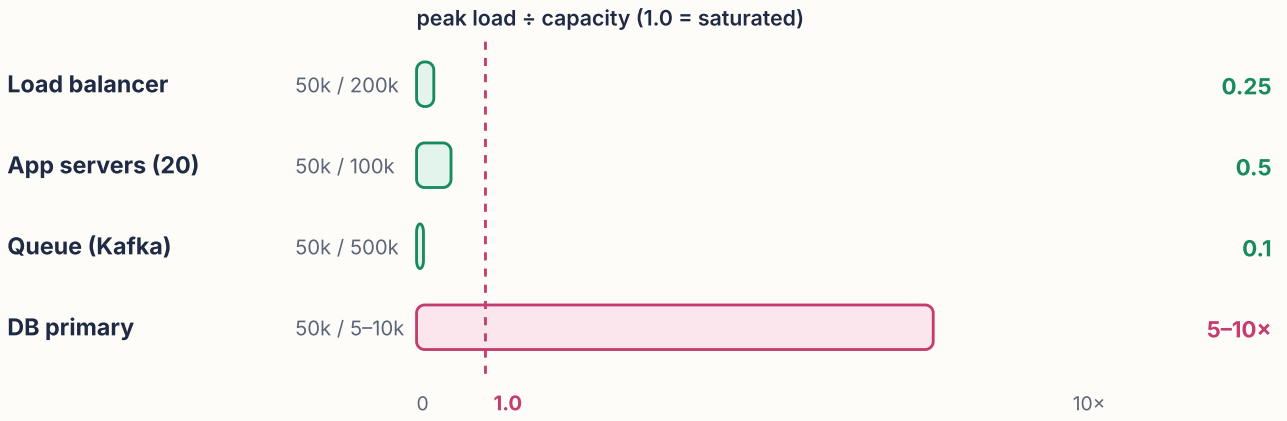


Figure 3. The workbook case at 50,000 writes/s peak. Capacities other than the database are illustrative. Only the primary is above 1.0, by 5 to 10 times.

WORKED EXAMPLE: 50,000 WRITES PER SECOND

Your estimates say **50,000 writes/s at peak**. A single relational primary on good hardware handles maybe **5,000–10,000** simple writes per second. So the write path is the bottleneck, by **5 to 10 times**.

The fixes:

- **Batch writes:** one insert of 500 rows costs far less than 500 inserts.
- **Buffer through a queue:** absorb the peak and write at a steady rate the database can take.
- **Shard** the database across about **10 primaries** (50,000 ÷ 5,000).
- **Move to a write-optimized store** like Cassandra, built for high write throughput.

5 Fixing one reveals the next

After sharding, maybe the queue consumers fall behind, or one shard runs hot. That is normal. Address bottlenecks **in order of impact**, and confirm them with **load tests and metrics**, not guesses. Say this out

loud in interviews: it shows you think about the system as a whole.

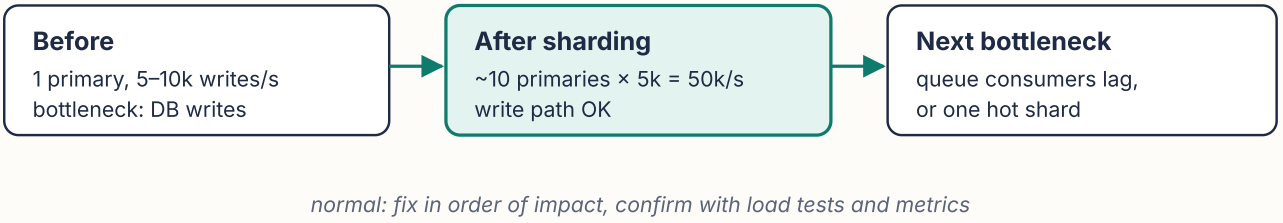


Figure 4. Bottlenecks move. Plan for the next one as soon as you fix the current one.

COMMON MISTAKES

- Adding app servers when the database is the limit: it just grows the queue in front of the database.
- Using average load instead of peak, and forgetting fan-out.
- Ignoring hot keys: total capacity is fine but one partition melts.
- Guessing the bottleneck instead of measuring it (USE, load tests).
- Stopping after one fix, without asking what breaks next.

6 Key terms

Bottleneck: the resource that limits whole-system throughput

Saturation: more work than capacity, so work waits

Hot partition: one key or shard taking far more load than others

Critical path: the components a request must pass through

USE method: utilization, saturation, errors per resource

Fan-out: one request turning into many downstream calls

7 Interview questions

Q1. What is a bottleneck? BASIC

The resource that limits throughput for the whole system. A path's throughput is the minimum capacity along it: a 5,000 writes/s primary caps the write path at 5,000 however many app servers you add.

Q2. How do you find the bottleneck in your design? BASIC

Walk the critical path of each request, write peak load and capacity next to every component, and divide. The highest ratio, the one closest to saturation, is the bottleneck.

Q3. What signals a bottleneck in production? BASIC

Saturation: CPU near 100%, disks maxed, an exhausted connection pool, or a queue whose depth keeps growing, with latency climbing sharply. Check each resource with USE.

Q4. Peak is 50,000 writes/s and one primary does 5–10k. What do you do? SCENARIO

It is 5–10× over. Batch and buffer writes through a queue, shard across about 10 primaries, or move to a write-optimized store like Cassandra. Then look for the next bottleneck.

Q5. Total traffic is fine but one shard is at 100%. Why, and what's the fix? SCENARIO

A hot partition: one key, like a celebrity account, gets far more traffic. Cache that key, split it across sub-keys (salting), or give it dedicated capacity.

Q6. After sharding, the queue's consumer lag keeps growing. What happened? SCENARIO

The bottleneck moved. Consumers now can't keep up. Add consumers and partitions (one consumer per partition in Kafka), or batch their writes.

Q7. A server is at 95% CPU but latency is fine. Is it the bottleneck? TRICKY

Not yet: high utilization without saturation can be healthy. But it has no headroom; a small spike will start queueing and latency will jump. Watch run-queue length, not just CPU %.

Q8. A feed request fetches 50 posts. Peak is 20,000 feed loads/s. What load does the post store see? TRICKY

Fan-out multiplies it: 20,000 × 50 = 1 million item reads/s, unless batched or cached. Fan-out is why the obvious tier is often not the bottleneck.

Next: SD-D-014 · Identifying SPOFs backendroadmap.com